

SECTION 2: RAG KNOWLEDGE ASSISTANT Q&A; REPORT

Project Evaluation & Out-of-Domain Grounding Verification

Student Name: SHRI RAM PRINCE MISHRA | Application No.: IN26012152 | GitHub:

https://github.com/srpm2005/HelpDeskManagement_IN26012152

Section 2A - Questions Answered from the Project Documentation

Below are 10 meaningful project questions asked to the RAG Knowledge Assistant and the exact AI-generated grounded responses with document provenance citations:

#	Question Asked	AI Generated Response & Citations
2A.1	What is the objective of the HelpDesk Ticket Management application?	### Answer Summary The objective of the Help Desk Ticket Management application is to provide an end-to-end enterprise solution for internal employees to submit, track, filter, and resolve IT support requests efficiently using ASP.NET Core 10.0 and SQL Server. #### ■ Context Provenance & Citations Answer synthesized from top matching snippets in: `RAG_KNOWLEDGE_ASSISTANT_REPORT.md` (Lines 1-1), `HelpDesk_Assignment_Report.md` (Lines 1-1), `HelpDesk_Assignment_Report.md` (Lines 6-11), `HelpDesk.Mvc/Models/Ticket.cs` (Lines 26-29) Citations: <i>RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L1-L1,</i> <i>HelpDesk_Assignment_Report.md#L1-L1,</i> <i>HelpDesk_Assignment_Report.md#L6-L11,</i> <i>HelpDesk.Mvc/Models/Ticket.cs#L26-L29</i>
2A.2	Which technologies were used to develop the project?	<p>### Answer Summary</p> <p>The project follows a clean 3-tier decoupled architecture:</p> 1. `HelpDesk.Api` (Web API Layer): Built with ASP.NET Core 10.0, EF Core, and SQL Server. Implements the Repository Pattern (<i>ITicketRepository`</i>, <i>TicketRepository`</i>). 2. `HelpDesk.Mvc` (MVC Frontend): ASP.NET Core MVC consuming the API strictly via <i>TicketService`</i> (<i>HttpClient`</i>). Follows a monochrome UI aesthetic. 3. `HelpDesk.Tests` (Unit Testing Layer): xUnit and Moq unit test project mocking <i>ITicketRepository`</i> with 100% isolated test runs.</li></ol> <p>#### ■ Context Provenance & Citations</p> <p>Answer synthesized from top matching snippets in:</p> <p>`HelpDesk.Mvc/appsettings.Development.json` (Lines 1-8), `HelpDesk.Api/appsettings.Development.json` (Lines 1-8), <i>README.md`</i> (Lines 103-113), <i>RAG_KNOWLEDGE_ASSISTANT_REPORT.md`</i> (Lines 39-49)</p> <p>Citations: <i>HelpDesk.Mvc/appsettings.Development.json#L1-L8,</i> <i>HelpDesk.Api/appsettings.Development.json#L1-L8,</i> <i>README.md#L103-L113,</i> <i>RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L39-L49</i></p>

2A.3	Which database tables are used in the application?	### Answer Summary The application utilizes Entity Framework Core with SQL Server ('SQLEXPRESS'). The primary database table is Tickets: <table><tr><th>Column Name</th><th>Data Type</th><th>Nullable</th><th>Description</th></tr><tr><td>Id</td><td>int</td><td>No</td><td>Auto-increment primary key</td></tr><tr><td>Title</td><td>nvarchar(100)</td><td>No</td><td>Short summary of support issue</td></tr><tr><td>Description</td><td>nvarchar(max)</td><td>No</td><td>Full details of support request</td></tr><tr><td>Priority</td><td>nvarchar(20)</td><td>No</td><td>Priority level ('Low', 'Medium', 'High')</td></tr><tr><td>Status</td><td>nvarchar(20)</td><td>No</td><td>Ticket status ('Open', 'In Progress', 'Closed')</td></tr><tr><td>RaisedBy</td><td>nvarchar(50)</td><td>No</td><td>Employee name raising the ticket</td></tr><tr><td>CreatedDate</td><td>datetime2</td><td>No</td><td>Timestamp of ticket creation</td></tr></table> #### ■ Context Provenance & Citations Answer synthesized from top matching snippets in: - RAG_KNOWLEDGE_ASSISTANT_REPORT.md (Lines 115-129), - RAG_KNOWLEDGE_ASSISTANT_REPORT.md (Lines 142-160), - README.md (Lines 103-113), - RAG_KNOWLEDGE_ASSISTANT_REPORT.md (Lines 130-141) Citations: RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L115-L129, RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L142-L160, README.md#L103-L113, RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L130-L141	Column Name	Data Type	Nullable	Description	Id	int	No	Auto-increment primary key	Title	nvarchar(100)	No	Short summary of support issue	Description	nvarchar(max)	No	Full details of support request	Priority	nvarchar(20)	No	Priority level ('Low', 'Medium', 'High')	Status	nvarchar(20)	No	Ticket status ('Open', 'In Progress', 'Closed')	RaisedBy	nvarchar(50)	No	Employee name raising the ticket	CreatedDate	datetime2	No	Timestamp of ticket creation
Column Name	Data Type	Nullable	Description																															
Id	int	No	Auto-increment primary key																															
Title	nvarchar(100)	No	Short summary of support issue																															
Description	nvarchar(max)	No	Full details of support request																															
Priority	nvarchar(20)	No	Priority level ('Low', 'Medium', 'High')																															
Status	nvarchar(20)	No	Ticket status ('Open', 'In Progress', 'Closed')																															
RaisedBy	nvarchar(50)	No	Employee name raising the ticket																															
CreatedDate	datetime2	No	Timestamp of ticket creation																															
2A.4	What validations are implemented while creating a ticket?	### Answer Summary The following validation rules are enforced: Title: Required, length between 3 and 100 characters. Description: Required, maximum length of 1000 characters. RaisedBy: Required, maximum length of 50 characters. Priority: Must be selected from Dropdown ('Low', 'Medium', 'High'). Default is 'Low'. Status: Hardcoded to 'Open' upon creation. Can be updated to 'In Progress' or 'Closed' during editing. #### ■ Context Provenance & Citations Answer synthesized from top matching snippets in: - RAG_KNOWLEDGE_ASSISTANT_REPORT.md (Lines 115-129), - HelpDesk.Api/Data/HelpDeskDbContext.cs (Lines 1-35), - RAG_KNOWLEDGE_ASSISTANT_REPORT.md (Lines 39-49), - RAG_KNOWLEDGE_ASSISTANT_REPORT.md (Lines 81-112) Citations: RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L115-L129, HelpDesk.Api/Data/HelpDeskDbContext.cs#L1-L35, RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L39-L49, RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L81-L112																																

2A.5	What are the different ticket statuses supported by the application?	### Answer Summary To run the Help Desk solution locally: 1. **Start Web API** (Port 5000): <pre>`dotnet run --project HelpDesk.Api/HelpDesk.Api.csproj --urls "http://localhost:5000"``</pre> </li> <li>2. <b>**Start MVC Web App**</b> (Port 5200):  <pre>`dotnet run --project HelpDesk.Mvc/HelpDesk.Mvc.csproj --urls "http://localhost:5200"``</pre> 3. Access the Dashboard at <code>`http://localhost:5200`</code>. #### ■ Context Provenance & Citations Answer synthesized from top matching snippets in: <code>`RAG_KNOWLEDGE_ASSISTANT_REPORT.md`</code> (Lines 115-129), <code>`HelpDesk.Mvc/Models/Ticket.cs`</code> (Lines 26-29), <code>`HelpDesk.Mvc/Controllers/TicketController.cs`</code> (Lines 176-210), <code>`HelpDesk.Mvc/Controllers/TicketController.cs`</code> (Lines 51-85) Citations: <code>RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L115-L129</code>, <code>HelpDesk.Mvc/Models/Ticket.cs#L26-L29</code>, <code>HelpDesk.Mvc/Controllers/TicketController.cs#L176-L210</code>, <code>HelpDesk.Mvc/Controllers/TicketController.cs#L51-L85</code>
2A.6	How does the MVC layer communicate with the Web API?	### Answer Summary The **HelpDesk.Mvc** application consumes the REST Web API via a dedicated Service Layer: - **Service Implementation**: <code>`HelpDesk.Mvc/Services/TicketService.cs`</code> encapsulates <code>`HttpClient`</code> calls. - **API Methods Executed**: - <code>`GetAllTicketsAsync()`</code>: Calls <code>`GetFromJsonAsync("<code>api/Ticket/All</code>")`</code> - <code>`GetTicketByIdAsync(id)`</code>: Calls <code>`GetFromJsonAsync("<code>api/Ticket/{id}</code>")`</code> - <code>`CreateTicketAsync(ticket)`</code>: Calls <code>`PostAsJsonAsync("<code>api/Ticket</code>", ticket)`</code> - <code>`UpdateTicketAsync(ticket)`</code>: Calls <code>`PutAsJsonAsync("<code>api/Ticket/{id}</code>", ticket)`</code> - <code>`DeleteTicketAsync(id)`</code>: Calls <code>`DeleteAsync("<code>api/Ticket/{id}</code>")`</code> - <code>`GetTicketsByStatusAsync(status)`</code>: Calls <code>`GetFromJsonAsync("<code>api/Ticket/Status/{status}</code>")`</code> MVC controllers interact strictly through <code>`TicketService`</code> with **zero direct database access**. #### ■ Context Provenance & Citations Answer synthesized from top matching snippets in: <code>`RAG_KNOWLEDGE_ASSISTANT_REPORT.md`</code> (Lines 130-141), <code>`HelpDesk.Mvc/Program.cs`</code> (Lines 1-31), <code>`RAG_KNOWLEDGE_ASSISTANT_REPORT.md`</code> (Lines 142-160), <code>`HelpDesk.Mvc/Controllers/TicketController.cs`</code> (Lines 101-135) Citations: <code>RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L130-L141</code>, <code>HelpDesk.Mvc/Program.cs#L1-L31</code>, <code>RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L142-L160</code>, <code>HelpDesk.Mvc/Controllers/TicketController.cs#L101-L135</code>

2A.7	What REST API endpoints exist in the Web API layer?	### Answer Summary The HelpDesk Web API (<code>HelpDesk.Api</code>) exposes standard RESTful endpoints for support ticket management: <table border="1"> <thead> <tr> <th>HTTP Method</th> <th>API Endpoint</th> <th>Description</th> </tr> </thead> <tbody> <tr> <td>GET</td> <td><code>/api/Ticket/All</code></td> <td>Retrieve all support tickets</td> </tr> <tr> <td>GET</td> <td><code>/api/Ticket/{id}</code></td> <td>Get ticket details by ID</td> </tr> <tr> <td>POST</td> <td><code>/api/Ticket</code></td> <td>Create a new support ticket</td> </tr> <tr> <td>PUT</td> <td><code>/api/Ticket/{id}</code></td> <td>Update existing ticket</td> </tr> <tr> <td>DELETE</td> <td><code>/api/Ticket/{id}</code></td> <td>Delete ticket by ID</td> </tr> <tr> <td>GET</td> <td><code>/api/Ticket/Status/{status}</code></td> <td>Filter tickets by status (Open/In Progress/Closed)</td> </tr> </tbody> </table> The controllers use <code>ITicketRepository</code> asynchronously for database operations. #### ■ Context Provenance & Citations Answer synthesized from top matching snippets in: <code>RAG_KNOWLEDGE_ASSISTANT_REPORT.md</code> (Lines 115-129), <code>README.md</code> (Lines 81-93), <code>RAG_KNOWLEDGE_ASSISTANT_REPORT.md</code> (Lines 130-141), <code>HelpDesk_Assignment_Report.md</code> (Lines 45-57) Citations: <i>RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L115-L129, README.md#L81-L93, RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L130-L141, HelpDesk_Assignment_Report.md#L45-L57</i>	HTTP Method	API Endpoint	Description	GET	<code>/api/Ticket/All</code>	Retrieve all support tickets	GET	<code>/api/Ticket/{id}</code>	Get ticket details by ID	POST	<code>/api/Ticket</code>	Create a new support ticket	PUT	<code>/api/Ticket/{id}</code>	Update existing ticket	DELETE	<code>/api/Ticket/{id}</code>	Delete ticket by ID	GET	<code>/api/Ticket/Status/{status}</code>	Filter tickets by status (Open/In Progress/Closed)
HTTP Method	API Endpoint	Description																					
GET	<code>/api/Ticket/All</code>	Retrieve all support tickets																					
GET	<code>/api/Ticket/{id}</code>	Get ticket details by ID																					
POST	<code>/api/Ticket</code>	Create a new support ticket																					
PUT	<code>/api/Ticket/{id}</code>	Update existing ticket																					
DELETE	<code>/api/Ticket/{id}</code>	Delete ticket by ID																					
GET	<code>/api/Ticket/Status/{status}</code>	Filter tickets by status (Open/In Progress/Closed)																					
2A.8	How are unit tests implemented and mocked using Moq?	### Answer Summary Unit testing is implemented in <code>HelpDesk.Tests</code>: - Built with <code>xUnit</code> and <code>Moq</code>. - Tests <code>TicketController</code> by mocking <code>ITicketRepository</code>. - All 13 test cases pass with zero SQL Server dependency. - Run command: <code>dotnet test HelpDesk.Tests/HelpDesk.Tests.csproj</code> #### ■ Context Provenance & Citations Answer synthesized from top matching snippets in: <code>HelpDesk.Tests/UnitTest1.cs</code> (Lines 1-10), <code>README.md</code> (Lines 94-102), <code>HelpDesk.Tests/Controllers/TicketControllerTests.cs</code> (Lines 1-35), <code>RAG_KNOWLEDGE_ASSISTANT_REPORT.md</code> (Lines 142-160) Citations: <i>HelpDesk.Tests/UnitTest1.cs#L1-L10, README.md#L94-L102, HelpDesk.Tests/Controllers/TicketControllerTests.cs#L1-L35, RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L142-L160</i>																					

2A.9	How to run the Web API and MVC applications locally?	### Answer Summary To run the Help Desk solution locally: 1. **Start Web API** (Port 5000): `dotnet run --project HelpDesk.Api/HelpDesk.Api.csproj --urls "http://localhost:5000"` 2. **Start MVC Web App** (Port 5200): `dotnet run --project HelpDesk.Mvc/HelpDesk.Mvc.csproj --urls "http://localhost:5200"` 3. Access the Dashboard at `http://localhost:5200`. #### ■ Context Provenance & Citations Answer synthesized from top matching snippets in: `RAG_KNOWLEDGE_ASSISTANT_REPORT.md` (Lines 130-141), `RAG_KNOWLEDGE_ASSISTANT_REPORT.md` (Lines 142-160), `RAG_KNOWLEDGE_ASSISTANT_REPORT.md` (Lines 115-129), `README.md` (Lines 103-113) Citations: <i>RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L130-L141, RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L142-L160, RAG_KNOWLEDGE_ASSISTANT_REPORT.md#L115-L129, README.md#L103-L113</i>
2A.10	What business rules govern editing ticket priority and status?	### Answer Summary The following **validation rules** are enforced: 1. **Title**: Required, length between 3 and 100 characters. 2. **Description**: Required, maximum length of 1000 characters. 3. **RaisedBy**: Required, maximum length of 50 characters. 4. **Priority**: Must be selected from Dropdown (`Low`, `Medium`, `High`). Default is `Low`. 5. **Status**: Hardcoded to `Open` upon creation. Can be updated to `In Progress` or `Closed` during editing. #### ■ Context Provenance & Citations Answer synthesized from top matching snippets in: `HelpDesk.Tests/Controllers/TicketControllerTests.cs` (Lines 1-35), `HelpDesk.Api/Repositories/TicketRepository.cs` (Lines 26-60), `HelpDesk.Tests/Controllers/TicketControllerTests.cs` (Lines 26-60), `HelpDesk.Mvc/Models/Ticket.cs` (Lines 1-29) Citations: <i>HelpDesk.Tests/Controllers/TicketControllerTests.cs#L1-L35, HelpDesk.Api/Repositories/TicketRepository.cs#L26-L60, HelpDesk.Tests/Controllers/TicketControllerTests.cs#L26-L60, HelpDesk.Mvc/Models/Ticket.cs#L1-L29</i>

Visual Screenshot Proof: RAG Answering Document Question with Citations



AI Knowledge Assistant

Retrieval-Augmented Generation (RAG) Developer Onboarding System

⚡ QUICK DEVELOPER ONBOARDING QUERIES

REST API Endpoints

Project Architecture

MVC to Web API Service

Unit Testing & Moq

Run Applications Locally

🔗 ASK QUESTION ABOUT CODEBASE

e.g. How does TicketController handle ticket creation?

🔍 Ask Assistant

SECTION 2B: OUT-OF-DOMAIN NON-DOCUMENT QUESTION TESTING

Student Name: SHRI RAM PRINCE MISHRA | Application No.: IN26012152

Section 2B - Questions Not Available in the Project Documentation

Below are 5 questions absent from the uploaded project documentation. This test evaluates the RAG system's hallucination prevention and strict grounding enforcement:

#	Non-Document Question	AI Response (Grounded Fallback)
2B.1	Who is the CEO of Microsoft?	The uploaded project documentation does not contain this information.
2B.2	What is Artificial Intelligence?	The uploaded project documentation does not contain this information.
2B.3	What is the capital of Australia?	The uploaded project documentation does not contain this information.
2B.4	Explain Machine Learning.	The uploaded project documentation does not contain this information.
2B.5	What is Cloud Computing?	The uploaded project documentation does not contain this information.

Visual Screenshot Proof: RAG Fallback Response for Non-Document Question

